CISC 322/326 Assignment 1

Conceptual Architecture of WezTerm

Oct 9, 2026

Authors:

Allen Liu

22xgp1@queensu.ca

Ted Pavlov

23ldn1@queensu.ca

Drew Ogilvie

23cjx2@queensu.ca

Michael Morris

23wwg3@queensu.ca

Ryder Tymon

21rtt1@queensu.ca

Aniso Kilas

24fsj@queensu.ca

Section 1: Abstract

WezTerm is a modular, event-driven terminal emulator whose architecture integrates terminal emulation, multiplexing, process execution, rendering, configuration, and remote connectivity through well-defined interfaces. Rather than following a strictly linear layered model, WezTerm combines modular and tiered architectural principles, allowing asynchronous events and data to move between interconnected components. The GUI serves as the primary input and presentation boundary, while the Multiplexer manages windows, tabs, panes, workspaces, and session state. The Domain and PTY layers abstract execution environments; allowing local processes, remote hosts, Unix sockets, and serial devices to all follow a common structure. At the centre of this design, the Terminal Emulator transforms byte streams into structured terminal state, while the Rendering System converts that state into graphical output using GPU-accelerated backends.

The architecture demonstrates its flexibility through practical use cases. Persistent multiplexer sessions allow a graphical client to disconnect while remote or local terminal sessions remain active, with client-side proxy objects reconstructing the remote session when the GUI reconnects. Similarly, serial-device support adapts physical communication into the existing domain and terminal abstractions, allowing subsystems and serial consoles to use a similar input, parsing, multiplexing, and rendering pipeline to that of conventional shells. These examples demonstrate how architectural boundaries reduce coupling while enabling different execution environments to share common functionality.

WezTerm's evolutionary potential is further supported by its Domain model, proxy objects, modular Rust components, and Lua-based configuration and extension mechanisms. New execution environments, protocols, and features can therefore be integrated without fundamentally redesigning the terminal emulator or rendering pipeline. This, crucially, makes the system more robust as it can evolve in a more seamless way. Overall, the architecture emphasizes separation of concerns, asynchronous event processing, abstraction, and extensibility, enabling WezTerm to function as both a conventional terminal emulator and a flexible platform for local, remote, multiplexed, and device-oriented terminal workflows.

TABLE OF CONTENTS

Section 1: Abstract **2**

Section 2: Architecture **4**

2.1: Architectural Principles 5

2.2: Component Analysis 5

2.2.1: GUI 5

2.2.2: Terminal Emulator 6

2.2.3: Rendering System 7

2.2.4: Multiplexer 7

2.2.5: Domain Layer 8

2.3: Component Interaction 8

Section 3: Use Cases **9**

3.1: Attach to a Persistent Mux Session and Type Remotely 9

3.2: Connect to a Serial Device 11

Section 4: How WezTerm evolves **133**

Section 5: External Systems **14**

Section 6 : AI Usage Report **14**

Section 7: Bibliography **17**

7.1: Citations 17

7.2: References 17

Section 2: Architecture

WezTerms architecture can be conceptualized as a modular architectural style; each component having well defined individual responsibilities, while communicating through well defined interfaces, similar to that of a layered style. The system thus combines modular architecture with elements of a layered style because terminal input, output, individual session events, and overall configuration changes must all be processed asynchronously, but still able to flow between the individual components. The layered aspect of the architecture is thus not strictly linear, borrowing more so from the tier aspect of a layered structure, where data flow between components within a tier is not strictly linear.

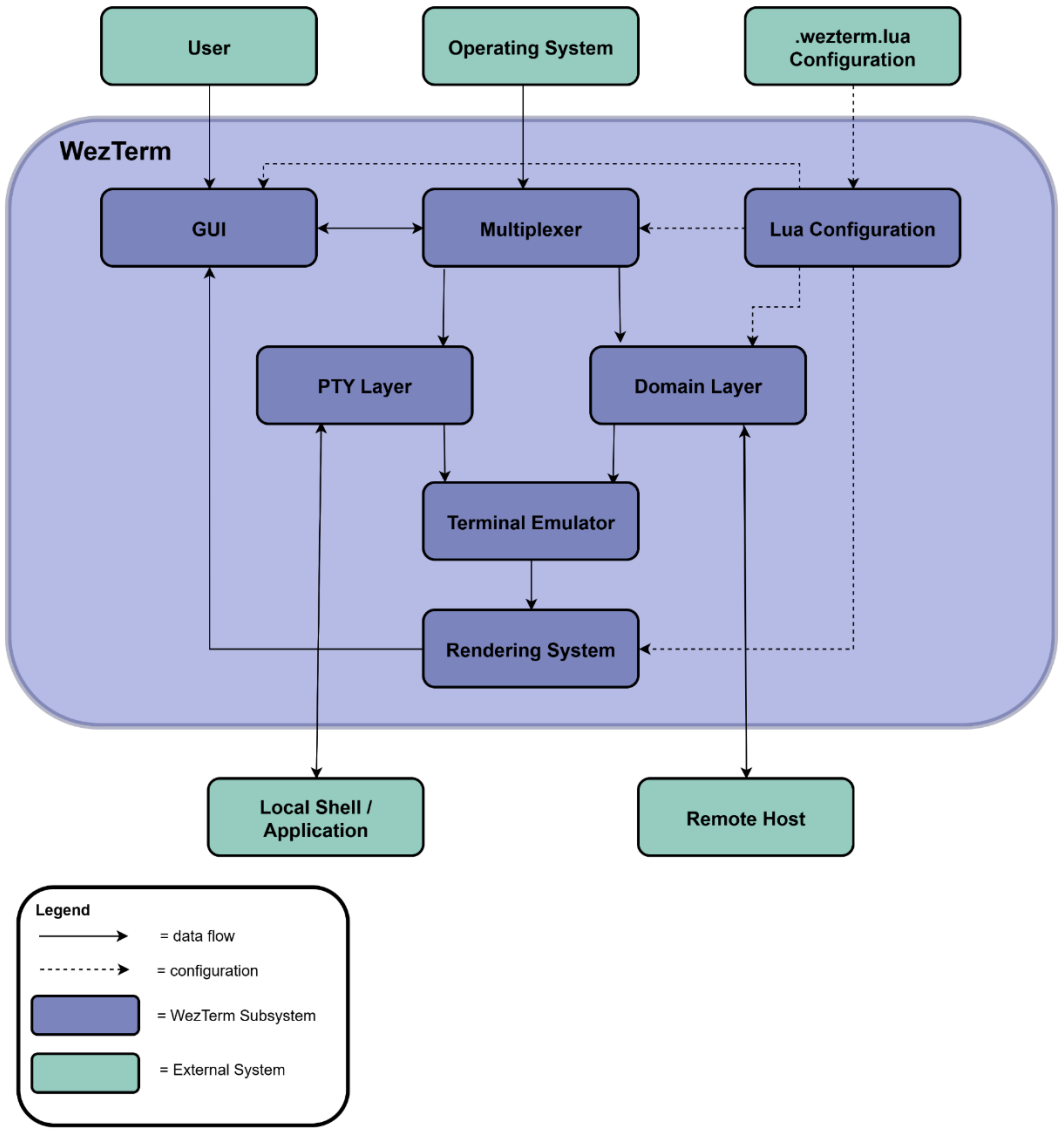


Figure 1: Conceptual Architectural Model for WezTerm

2.1: Architectural Principles

The GUI, Multiplexer, PTY Layer, Domain Layer, Terminal Emulator, and Rendering System form an interconnected pipeline through which commands and terminal data flow. The Lua Configuration layer operates somewhat differently because it acts as a configuration and extension mechanism across several components rather than being a conventional processing stage. The architecture is also event-driven. User actions generate events that propagate through the system, flowing through components as needed using implicit invocation through causing a change in terminal state, which various components react to before ultimately triggering the rendering of the final result.

The reason this is the preferred architectural style is to limit the load on each component. The Multiplexer does not need to understand how pixels are rendered, the Renderer does not need to understand shell processes, and the PTY layer does not need to understand panes or workspaces. Similarly, the Terminal Emulator interprets terminal semantics without being responsible for the physical connection to the shell. This serves two purposes; primarily, a malfunction in the system is easily traceable due to no sharing of functionality between components being very limited, and it simply makes things more efficient when every component does exactly what they were engineered to do. One potential issue when put into practice, however, is duplication of processes, as a component may later need something another component already did, but as it lacks access to that information, it would be forced to repeat that process. It is therefore important to know exactly what information to communicate between components to mitigate this issue.

This system also allows the architecture to evolve easily in the future; data crosses boundaries in well-defined forms: input events, multiplexer commands, process I/O, terminal state, rendering information, and configuration data. Additionally, the architecture separates control flow from data flow. Control operations such as creating a pane or selecting a domain originate at the GUI and move toward the Multiplexer, while terminal data generated by applications moves back toward the Terminal Emulator and Rendering System. This allows individual subsystems to evolve independently. It must, however, be ensured that output from each component is backwards compatible with the rest of the system, else the entire system must change to accommodate the new component, losing this advantage.

2.2: Component Analysis

2.2.1: GUI

The GUI is the primary presentation and input subsystem. It represents the boundary between the human user and the internal WezTerm architecture. Its responsibilities include creating and managing the visible application interface, receiving keyboard and mouse events, determining which terminal surface currently has focus, and presenting rendered terminal content in windows, tabs, and panes. From an architectural perspective, the GUI acts as an event source and presentation layer. Physical keyboard and mouse activity is converted into higher-level application events. For example, a mouse click may result in a focus change, pane selection, text-selection

operation, or multiplexer command. Keyboard input may represent ordinary terminal data or an application-level shortcut.

The GUI communicates with the Multiplexer when an event affects session organization. A request to create a pane, close a tab, switch a workspace, or resize a pane is conceptually routed to the Multiplexer because those operations modify session state rather than merely changing pixels on screen. The GUI also interacts with the Terminal Emulator for terminal-related operations. This distinction is important because not every keyboard or mouse event represents terminal input. Some events are consumed by WezTerm itself, while others must eventually be delivered to the running terminal application. The GUI receives visual output from the Rendering System. The Renderer transforms terminal state into graphical primitives, while the GUI provides the windowing context in which those primitives are displayed.

The GUI therefore maintains a separation between input handling, window management, terminal presentation, and session control. It should not need to know how a shell process is created or how a remote connection is established. Lua Configuration can modify GUI-level properties such as appearance, key bindings, mouse behaviour, fonts, colours, and other interface-related settings [4]. Consequently, the GUI is both a presentation subsystem and a configurable event-processing boundary.

2.2.2: Terminal Emulator

The Terminal Emulator is the semantic core of the architecture. In the actual WezTerm source tree, the wezterm-term crate provides the core virtual-terminal implementation. It explicitly provides escape-sequence parsing, keyboard and mouse input encoding, screen-cell modelling, scrollback, Sixel and iTerm2 image support, OSC 8 hyperlinks, and terminal cell attributes [1]. It deliberately does not provide a GUI or directly manage a PTY. WezTerm communicates using a byte stream. The terminal emulator converts that stream into state changes. A control sequence might modify cursor position, screen contents, text attributes, scrolling regions, hyperlinks, images, or terminal modes. This therefore causes the terminal to behave like a state machine rather than a mere text editor.

The source architecture is particularly interesting because the terminal model can accept an abstract std::io::Write implementation connected to a PTY or another transport. This means the terminal core is not intrinsically coupled to local process execution. That abstraction is essential to WezTerms Domain model. The same terminal-emulation machinery can process output originating from different execution environments, and is what allows WezTerms terminal implementation to support advanced graphical terminal features such as Sixel and iTerm2 image protocols.

2.2.3: Rendering System

The Rendering System converts logical terminal state into graphical output. It is intentionally separated from the terminal state machine, meaning terminal semantics and graphical presentation are distinct subsystems. WezTrem supports GPU rendering through OpenGL and WebGPU-based backends. Its documentation identifies OpenGL as a GPU-accelerated rasterization path and WebGPU as another GPU-accelerated option, with platform-specific backends including Metal, Vulkan, and DirectX 12.

The way the rendering system works, at least conceptually, is by starting with the internal terminal state, which was just updated by other components, and uses glyph processing to parse the information for stylized rendering such as the layout and stylized font. It then uses the GPUs render data and the systems graphics backend to render the text in the window. The Renderer therefore operates on structured terminal state rather than raw terminal byte streams. This distinction improves modularity because the terminal parser can change independently of the graphics backend. It also permits platform specific GPU APIs without requiring the terminal-emulation layer to understand graphics APIs, allowing this aspect to be platform agnostic.

The rendering is also done in multiple passes; the project's technical discussion describes separate passes for backgrounds, glyphs, and decorations such as underlines and strikethroughs, with GPU blending used to efficiently compose these layers. This becomes particularly important when rendering multiple panes because the renderer must combine independent terminal surfaces into a single window while respecting pane geometry, backgrounds, transparency, text, and overlays, which can prove difficult to do without sacrificing throughput and latency. Latency in particular is important to consider for the average user, as having to wait too long for input to be processed is a huge deterrent for most users.

2.2.4: Multiplexer

The Multiplexer is arguably the most architecturally distinctive component in the diagram because WezTerm does not rely on an external multiplexer for their terminal emulator. Multiplexing is represented internally as a domain-aware model. A design document in the WezTerm repository describes the Mux model as maintaining sets of domains and panes, with windows containing ordered tabs and tabs maintaining a binary-tree representation of pane splits. It documents explicit Mux structures for domains, panes, windows, tabs, clients, notifications, and related state [2].

Every pane in Wezterm corresponds to a terminal resource represented in the Mux model. This is a significant architectural distinction from traditional terminal emulators. The Multiplexer owns the logical topology of terminal resources, while the GUI provides one possible visualization of that topology. The Mux also exposes notifications and domain lookup mechanisms in its Rust implementation, supporting event-driven changes to session state [2]. This integration is WezTerms primary selling point; it allows users to combine multiple widgets into one workflow.

2.2.5: Domain Layer

The Domain Layer is one of the most technically significant parts of WezTerm because it abstracts the location and mechanism of terminal execution. A domain represents an execution context that the Multiplexer can interact with without requiring the Mux itself to understand every transport mechanism, which as discussed leads to a very dynamic system that is easy to update. WezTerm supports several domain concepts, including local domains, Unix domains, SSH domains, TLS domains, and more. The official documentation describes SSH domains in which WezTerm connects to a remote machine and starts or connects to a remote WezTerm multiplexer. Unix domains can connect a GUI to a multiplexer through a Unix socket and are also used for scenarios such as Windows/WSL integration. TLS domains provide a TLS-protected TCP connection to a multiplexer [3].

The remote multiplexer can therefore maintain the session independently of the local graphical representation. WezTerm also embeds an SSH implementation, allowing multiple tabs or panes to use channels within an existing SSH session rather than requiring independent authentication for every new terminal. However, ordinary ad-hoc SSH sessions are non-persistent; the documentation specifically distinguishes them from multiplexing domains that preserve remote terminal state [3]. This is a major distinction between WezTerm and its competitors; allowing one terminal structure to execute locally or remote as needed.

2.3: Component Interaction

As outlined in Figure 1 above, the complete architecture can be understood as a bidirectional event and data-processing system rather than a simple top-to-bottom stack. In the first direction of the control path, when the user creates or modifies a pane, tab, or workspace, the GUI generates a control operation for the Multiplexer. The Multiplexer updates its internal model of windows, tabs, panes, and domains. If a new pane requires a local shell, the Multiplexer selects the local domain and the process is connected through the PTY layer. If it requires a remote environment, the Domain Layer establishes or uses the appropriate remote connection. This is where WezTerm's domain architecture provides a major architectural benefit; the GUI and Mux do not fundamentally need different session-management logic for every execution location.

The second direction almost like a feedback loop, but uses a flow through other components to reach the user; the shell or application produces bytes. The PTY or Domain transports those bytes. The Terminal Emulator parses them and updates the virtual terminal model. The Renderer then converts this logical state into graphics. GPU acceleration allows the graphical representation to be generated efficiently, with WezTerm supporting OpenGL and WebGPU rendering paths. At the end of this pseudo feedback loop, that process simply terminates, as multiple concurrent processes are running, with a new one invoked with every event received. This is facilitated by the independence of components, allowing them to start working on another process without waiting on the other components to finish theirs.

Section 3: Use Cases

3.1: Attach to a Persistent Mux Session and Type Remotely

This use case demonstrates how WezTerm separates the GUI client from the persistent multiplexer server, allowing a user to disconnect the graphical interface while the terminal sessions and shells continue running. The architecture begins when the user executes wezterm connect unix. A .wezterm.lua configuration containing config.unix_domains = { { name = "unix" } } defines a Unix multiplexer domain. WezTerm documentation describes Unix domains as connections to a multiplexer through a Unix socket, with the client able to automatically start the server when required [3]. The GUI initially creates its own local Mux representation and registers the configured ClientDomain. The GUI then calls attach(), causing the Mux client to establish a Unix-domain socket connection to the persistent wezterm-mux-server. If the server is absent and automatic serving is enabled, WezTerm can launch the mux server as a daemon and retry the connection. This is an important architectural distinction: the GUI process is a client, while the server process owns the persistent terminal state.

The connection begins with a protocol handshake. The client requests the server's codec version and compares it with its own supported protocol version. If compatible, the client establishes its identity and requests the server's current pane hierarchy. The server is authoritative: it returns its windows, tabs, panes, and split-tree structure. The client then constructs ClientPane proxy objects representing those remote panes locally. This allows the GUI to interact with remote panes through the same conceptual Pane interface used for local panes. This abstraction is central to WezTerm's architecture. Its documentation explicitly states that attaching a domain imports the remote system's windows, tabs, and panes into the local GUI [3]. The GUI therefore does not need a separate rendering architecture for remote sessions; a remote pane is represented locally and rendered through the normal terminal UI.

When the user presses a preconfigurable key, the keyboard event enters the GUI event loop and is processed by TermWindow. Because l is not consumed by a key binding, it becomes terminal input and is passed to the ClientPane. The ClientPane can use predictive local echo when network round-trip latency exceeds local_echo_threshold_ms. WezTerm predicts the character locally and associates it with an input serial number, allowing the prediction to be reconciled when authoritative server state arrives. The documentation specifically identifies this mechanism as a way of hiding network latency. The keystroke is then transmitted asynchronously to the server as a protocol message. Crucially, the GUI does not synchronously wait for the remote shell. The server's session handler delivers the input to the real server-side pane, whose PTY forwards it to the shell. Output travels in the opposite direction. The shell writes to the PTY, the server-side terminal parser updates the terminal state, and the Mux publishes a PaneOutput notification. Rather than repeatedly transmitting the entire terminal buffer, the server computes render changes/deltas, which the client applies to its cached RenderableState. The client then publishes the update locally, causing TermWindow to invalidate and repaint. The renderer can read the cached visible lines without performing network I/O during painting.

This architecture provides a persistent-session model similar to tmux, but with a key difference: WezTerm integrates the multiplexer directly with its native GUI, mouse handling, scrollback, rendering, and configuration system. The trade-off is that tmux has a more mature, terminal-independent server/client ecosystem, while WezTerm's tighter integration reduces the impedance between remote session state and the graphical terminal [10]. Finally, closing the GUI does not terminate the server-side panes or shells. The socket disappears, but the persistent Mux remains active. Re-running wezterm connect unix reconstructs the client-side representation and attaches to the existing state, demonstrating the fundamental principle of this use case: the GUI is disposable; the multiplexed execution environment is persistent.

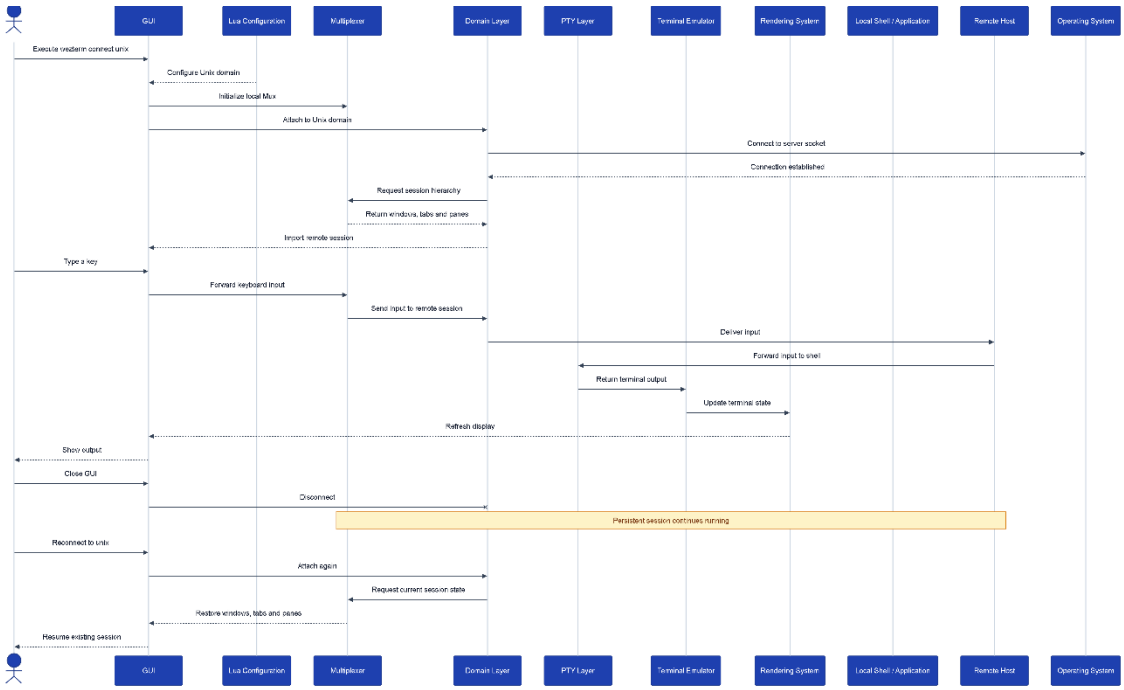


Figure 2: Sequence Diagram for Attaching to a Persistent Mux Session

3.2: Connect to a Serial Device

This use case demonstrates how WezTerm extends its terminal architecture beyond conventional shells and PTYs to support direct serial communication with embedded devices, Arduino boards, and serial consoles. WezTerm officially supports serial connections through the wezterm serial command, including configurable baud rates, and its documentation specifically identifies embedded-device and headless-server console access as use cases.

The architecture begins when the user executes wezterm serial --baud 115200 COM3, or the equivalent Unix command using a device such as /dev/ttyUSB0. The CLI delegates the operation to the GUI process, which performs its normal configuration initialization. This means serial mode still participates in WezTerm's broader configuration architecture: fonts, colours, key bindings, rendering preferences, and other settings can be supplied through wezterm.lua. WezTerm evaluates the Lua configuration as part of application startup, and configuration can also be reloaded dynamically [4].

The serial-specific configuration is represented explicitly by serial_ports. Each entry defines a serial domain with a name, device path, and baud rate; the documented default baud rate is 9600 when one is not specified. Internally, the configuration model exposes serial_ports alongside other domain types such as Unix, SSH, WSL, and execution domains, demonstrating that serial communication is integrated into the same domain-oriented architecture rather than being implemented as an isolated terminal mode.

The key architectural decision is that the serial device is adapted into WezTerm's existing LocalDomain/PTY abstraction. Instead of forcing the Multiplexer to understand serial-specific operations, a SerialTty provides the underlying terminal I/O implementation. Consequently, the Multiplexer can create a pane using essentially the same lifecycle used for ordinary terminal execution. The important difference is that no shell or child process exists on the other side of the connection. The serial port itself becomes the external I/O endpoint.

This design substantially reduces coupling. The terminal emulator does not need to know whether its bytes originated from Bash, PowerShell, an SSH session, or an Arduino. It receives a byte stream and applies terminal semantics to it. WezTerm's feature documentation explicitly lists serial-port support alongside its normal terminal, multiplexing, SSH, and graphical capabilities.

Once the port is opened, the normal terminal pipeline becomes active. User keyboard input enters the operating-system window event loop and is translated into WezTerm's platform-independent key-event representation. TermWindow checks configured key bindings before passing ordinary input to the active pane. WezTerm's key bindings are configuration-driven and can be inspected using wezterm show-keys.

The input then passes through the terminal model and is encoded into bytes before being handed to the serial writer. At 115200 baud, these bytes travel across the serial TX line to the

embedded device. Conversely, the device can asynchronously transmit data at any time—for example, boot diagnostics, sensor readings, a firmware log, or a command prompt. The important property is that serial I/O does not occur directly on the GUI thread. Input and output can proceed concurrently, while parsing and rendering remain separate stages. This prevents slow serial communication or device output from directly blocking graphical event processing.

After received bytes reach the terminal model, they are interpreted as terminal data and control sequences. Ordinary characters update the screen grid, while escape sequences can modify cursor position, colours, scrolling behaviour, and other terminal state. The Mux then notifies the GUI that pane output has changed. The GUI invalidates the affected display region and eventually repaints it through WezTerm's GPU rendering pipeline.

An interesting architectural consequence is that window resizing does not have the same meaning as with a conventional PTY. A shell connected to a PTY can be told that its terminal changed from, for example, 120×30 to 160×40 cells. A physical serial device does not inherently have a terminal geometry. Therefore, resizing primarily affects the local terminal presentation rather than communicating a new terminal size to the embedded device. Serial domains are also deliberately constrained. WezTerm documents that serial ports are represented by serial domains that are not spawnable, because there is no child process to launch inside the domain. This illustrates how the Domain abstraction accommodates fundamentally different execution environments without pretending that they have identical semantics.

Finally, disconnect detection requires another adaptation. A normal local pane can determine that its process has terminated by waiting for the child process. A serial connection has no equivalent process exit event. Instead, the serial implementation can monitor the physical connection state, such as carrier detection, and translate loss of the connection into the pane lifecycle. The Mux can then remove the dead pane and allow the configured exit behaviour to close the corresponding tab or window.

Overall, this use case demonstrates one of WezTerm's strongest architectural properties: the terminal emulator, multiplexer, configuration system, event loop, and renderer are sufficiently decoupled that a physical serial device can be inserted beneath the existing terminal abstraction without requiring a separate serial-specific GUI or terminal implementation. Serial support consequently reuses the same architecture rather than creating a parallel application path. This is consistent with WezTerm's broader design around domains: different execution mechanisms can be presented through a common terminal and multiplexer interface.

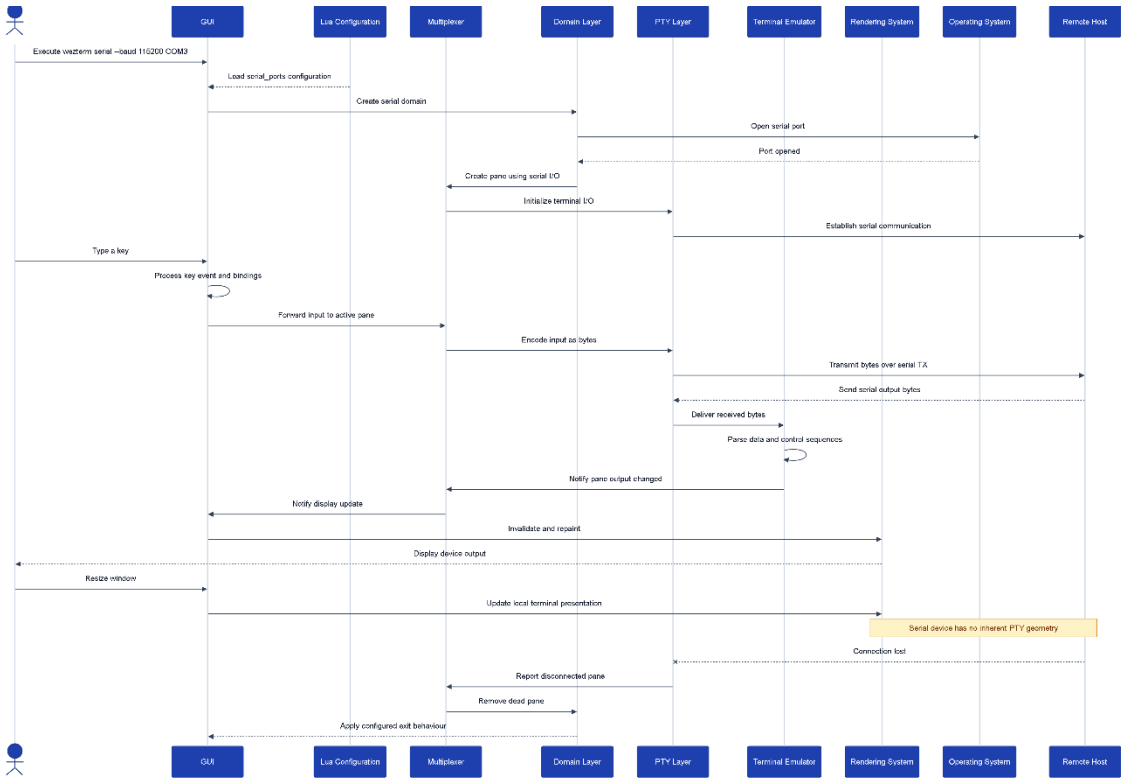


Figure 3: Sequence Diagram for Attaching External Device to Serial Port

Section 4: How WezTerm evolves

As briefly touched on previously, WezTerms architecture supports evolution by separating terminal semantics, process execution, multiplexing, rendering, GUI behaviour, and configuration into distinct layers. This modularity means new capabilities can be introduced by extending an existing abstraction rather than redesigning the entire application. The Domain model is a strong example: local processes, SSH connections, Unix sockets, WSL environments, execution domains, and serial devices can be represented through related domain abstractions while the higher-level Multiplexer continues to work with panes, tabs, windows, and workspaces.

The Mux architecture is also explicitly designed around replaceable and proxyable objects. WezTerm can create proxy Domain and Pane objects when connecting to a remote multiplexer, allowing remote state to appear through the same local interfaces. The project's Mux design documentation describes this model and its evolution toward workspaces and explicit client tracking.

Lua provides a second evolution mechanism outside the compiled core. Configuration can modify behaviour, access Mux functionality, define custom execution domains, and load plugins without modifying Rust source code [4], [6]. WezTerm plugins are distributed as Lua packages, providing an extension mechanism for functionality that does not need to become part of the core application.

Finally, WezTerm's architecture is actively evolving: its documentation describes multiplexing as a feature that is still developing rapidly [3]. This combination of modular Rust components, abstraction layers, domain-based execution, proxy objects, and Lua extensibility allows new platforms, protocols, device types, and user-facing functionality to be incorporated while preserving the core terminal and rendering pipeline.

Section 5: External Systems

WezTerm relies on remote hosts, which are servers and machines that WezTerm connects to through SSH. WezTerm sends encrypted user input and receives the remote program's output. When a WezTerm multiplexer runs on the remote host, session state can remain on that machine while the local GUI connects and disconnects as required [3].

WezTrem also relies on local shells such as Bash, Zsh, and PowerShell run through a pseudo-terminal (PTY) created by WezTerm. The PTY provides a two-way communication channel: WezTerm sends keyboard input to the shell and receives the shell's terminal output for processing and display.

One of WezTerms unique selling points is the connection to serial devices, and those are also external system WezTerm thus relies on. Serial devices include Arduino boards, embedded systems, routers, and serial consoles connected through ports such as COM3 or /dev/ttyUSB0. WezTerm exchanges raw bytes with the device at the configured baud rate, transmitting user input and displaying data received from the device.

Finally, WezTerm uses GPU drivers to provide the interface between WezTerm's rendering system and the graphics hardware through APIs such as OpenGL or WebGPU. WezTerm submits rendering commands containing glyphs, colours, images, and layout information, which the graphics subsystem converts into pixels displayed in the terminal window.

Section 6 : AI Usage Report

Our group chose to use Claude Sonnet 5.5 (Anthropic, September 2026) and OpenAI ChatGPT (GPT-5.6 Luna, September 2026) as our virtual teammates. Claude was chosen for work involving sequence diagrams because of its ability to render the diagram within itself. This allowed the person prompting it to make adjustments by simply telling Claude what needed to be changed, rather than having to repeatedly modify and render the Mermaid syntax themselves. ChatGPT was chosen mainly for research assistance because of its fact-finding and source-comparison capabilities. It was also used for consistency assurance because it could compare different areas of our report and identify inconsistencies or mistakes.

The main tasks we delegated to our AI teammates were construction of sequence diagrams, research assistance, and consistency assurance. Claude was used to construct the sequence diagrams because writing Mermaid syntax can be tedious, while the AI could format the syntax according to our instructions. We did not rely on Claude to design the diagrams itself, we determined what the diagram should contain and instructed it to represent those components and relationships. ChatGPT was used for research because finding and comparing sources for our report was time-consuming. Having AI search for different sources and compare them helped shorten this process, although we still verified the information and did some research of our own. ChatGPT was also used for consistency assurance, particularly when comparing different sections of our report to make sure that they were describing the same conceptual architecture.

Each group member who used AI was allowed to create and refine their prompts as they saw fit. We would iterate on prompts by changing or adding information to make the AI's output better match what we were looking for. For example, one research prompt used with ChatGPT was: "We are developing a conceptual software architecture for WezTerm. Identify use cases of WezTerm, document your source for each use case, and focus on important things that the user specifically will use." We then refined the research by asking it to "Prioritize use cases that are unique to WezTerm and are integral to the utilization of WezTerm as a user." The results were reviewed by two group members, and two use cases were eventually chosen to be highlighted in the report. When constructing sequence diagrams, we also clarified to Claude that the diagram was to be constructed based on our instructions, and that it should not add things beyond what we requested.

All facts, sources, claims, and summaries provided by the AI teammates were looked over by at least two group members, and typically more, to make sure the information was accurate and made sense. Sources provided by the AI were always investigated, and we located the relevant information ourselves to check for missing context or information that could change the interpretation or relevance of the AI's response. Any suggested adjustments to the wording of a section were also reviewed by the people responsible for the affected sections. This helped ensure that changing one part of the report did not create inconsistencies elsewhere.

The sequence diagrams went through a similar process. Each iteration was checked by the person who prompted Claude to make sure it contained all of the requested components and relationships, without adding unnecessary ones. When this was not the case, the prompt was clarified and the diagram was generated again. If Claude added or removed something unexpectedly, we also considered whether the AI had made a mistake or whether our original instructions were unclear.

We estimate that the Al teammates contributed roughly 10% of the total workload. The group was still responsible for most of the work, while Al mainly helped with the more repetitive or time-consuming tasks, such as creating Mermaid diagrams, sorting through sources, and checking consistency. Al involvement saved time in these areas without creating a large amount of additional work through prompting and validation. It also helped us compare options when deciding which use cases and evolution points to elaborate on and helped identify information in the documentation that could otherwise have been missed.

Overall, we learned that Al can be useful for reducing menial work and helping with research and comparison, but its output still needs to be checked carefully. Sometimes getting a useful response required multiple prompt iterations, and information that appeared reasonable still had to be checked against the original sources. For future work, we would continue using Al for these types of tasks while keeping the actual decisions and verification with the group.

Section 7: Bibliography

7.1: Citations

[1] WezTerm Contributors, “Cargo.toml,” wezterm/wezterm repository, GitHub. Accessed: Oct. 5, 2026. [Online]. Available: https://github.com/wezterm/wezterm/blob/main/Cargo.toml

[2] WezTerm Contributors, “mux/src/lib.rs,” wezterm/wezterm repository, GitHub. Accessed: Oct. 5, 2026. [Online]. Available: https://github.com/wezterm/wezterm/blob/main/mux/src/lib.rs

[3] WezTerm Contributors, “Multiplexing,” WezTerm Documentation. Accessed: Oct. 5, 2026. [Online]. Available: https://wezterm.org/multiplexing.html

[4] WezTerm Contributors, “Configuration Files,” WezTerm Documentation. Accessed: Oct. 5, 2026. [Online]. Available: https://wezterm.org/config/files.html

[5] WezTerm Contributors, “Lua Reference,” WezTerm Documentation. Accessed: Oct. 5, 2026. [Online]. Available: https://wezterm.org/config/lua/general.html

[6] WezTerm Contributors, “wezterm.mux module,” WezTerm Documentation. Accessed: Oct. 5, 2026. [Online]. Available: https://wezterm.org/config/lua/wezterm.mux/index.html

7.2: References

[7] Ghostty Contributors, “ghostty-org/ghostty,” GitHub. Accessed: Oct. 5, 2026. [Online]. Available: https://github.com/ghostty-org/ghostty

[8] K. Goyal, “kitty documentation,” kitty repository, GitHub. Accessed: Oct. 5, 2026. [Online]. Available: https://github.com/kovidgoyal/kitty/blob/master/docs/index.rst

[9] K. Goyal, “Terminal graphics protocol,” kitty documentation, GitHub. Accessed: Oct. 5, 2026. [Online]. Available: https://github.com/kovidgoyal/kitty/blob/master/docs/graphics-protocol.rst

[10] tmux Contributors, “Getting Started,” tmux Wiki, GitHub. Accessed: Oct. 5, 2026. [Online]. Available: https://github.com/tmux/tmux/wiki/Getting-Started/c934355b542af8289ef153db7efbf933364045aa

[11] Microsoft, “Windows Terminal rendering settings,” Microsoft Learn. Accessed: Oct. 5, 2026. [Online]. Available: https://learn.microsoft.com/en-us/windows/terminal/customize-settings/rendering